Search docs...

GETTING STARTED
Overview
Authentication
Conventions & limits
Data usage & license FREE

GUIDES
Quickstart: first request
Build a league page
Football player profile
Tennis player profile
Connect to live WebSockets
Working with images
Use BSD from AI agents

FOOTBALL API
Overview FREE
Events & live scores
Leagues, seasons & standings
Competition structure FREE
Teams, players & transfers
Managers, referees & venues
Odds & predictions
TV, broadcasts & social
v1 API (legacy)

MORE SPORTS APIS
Tennis API PRO
Padel API PRO
CS2 API PRO
Darts API PRO
Hockey API PRO
Basketball API PRO
Horse Racing API PRO

WEIGHT OF MONEY
Weight of Money API ADDON

WEBSOCKETS
Overview ADDON
Football live channel
Tennis live channel
Arena 3D

AI & AGENTS
MCP servers

IMAGES & DATA
Image API
Static datasets
OpenAPI explorer

LEGAL
Terms of Service

Events & live scores#

An **event** is one match. The list endpoint answers "which matches", the detail and sub-resources answer "everything about this match".

ON THIS PAGE
List matches
Live matches
Match detail
Head-to-head
Measured xG and estimated xG
Shot counts: which number to trust
Odds, and when they refresh
Sub-resources
Head-to-head
Measured xG and estimated xG
Shot counts: which number to trust
Odds, and when they refresh
Stats coverage varies
When a match is postponed
Which line is the main line
Fixture change feed
Ask before you fetch
Coverage on past matches
When a lineup exists
Coverage on past matches
Predicted benches
Typical patterns

List matches

```
GET /api/v2/events/
```
copy

PARAM	TYPE	DESCRIPTION
`league_id`	int	Filter by league
`season_id`	int	Filter by season (use for "this season's fixtures")
`team_id`	int	Matches of one team (home or away)
`team_name`	string	Fuzzy team-name filter when you don't have the id
`status`	string	`upcoming` · `live` · `finished` · `cancelled` · `postponed` · `unresolved`
`date_from` / `date_to`	date	`YYYY-MM-DD` window (inclusive)
`stage`	string	Competition stage, e.g. `league-phase`, `group-stage`, `round-of-16`. See Competition structure
`round`	int	Matchday within `stage`. Requires `stage` — round numbers repeat across stages
`limit` / `offset`	int	Pagination, default 50, max 200

```
curl -H "Authorization: Token YOUR_API_KEY" \
  "https://sports.bzzoiro.com/api/v2/events/?league_id=85&status=upcoming&limit=10"
```
copy

Live matches

```
GET /api/v2/events/live/
```
copy

Optional `league_id`, `season_id`, `team_id`. Returns a compact live shape — score, period, `current_minute`, half-time score, extra time and penalty shootout state — plus two capability flags:

`home_score` / `away_score` are **regulation time only**. Goals scored in extra time arrive in `extra_time_score` and a shootout in `penalty_shootout`, both `null` until they apply — so a match in extra time shows an unchanged 90-minute scoreline unless you add the two together.

- `live_websocket` — this match can be followed on the WebSocket
- `websocket_plus` — full per-action coverage (individual events with pitch coordinates) is available for it

Server-cached ~10–30 s; poll at 10 s or subscribe to the WebSocket.

Match detail

```
GET /api/v2/events/{id}/
```
copy

Everything static about the match: teams (ids + names), coaches, referee, venue, round/group, kickoff time, scores (FT/HT/ET/pens), weather, pitch condition, attendance, derby/neutral-ground flags, travel distance, highlights and a head-to-head summary. Also `has_xg` (whether shot-level xG exists for this match) and `previous_leg_event_id` — set when the match is the second leg of a two-legged knockout tie, pointing at the first leg's event id.

Sub-resources

Every deeper view of a match is its own endpoint under `/api/v2/events/{id}/…`:

ENDPOINT	RETURNS
`/stats/`	Team stats (possession, shots, xG — per half where available), per-shot **shotmap**, momentum graph, average player positions, xG-per-minute series. See how much of it you actually get
`/lineups/`	Confirmed XI + bench, each player flagged `captain: true/false`; or an AI-predicted XI with confidence scores before kickoff; or nothing at all — branch on `lineup_status`. See when a lineup exists and predicted benches
`/incidents/`	Chronological goals, cards, substitutions, VAR decisions — each stamped with `period_second` for sub-minute ordering. Cards can carry `rescinded: true` when a card is overturned on review; overturned cards stay in the timeline but are excluded from card counts
`/player-stats/`	Per-player match statistics, including an `Advanced` category (big chances, xG on target, ball-carry progression, goalkeeper sweeper actions)
`/h2h/`	Past meetings, W/D/L, goals, win rates, recent results — see Head-to-head
`/odds/`	Consensus decimal odds per market, plus when they next refresh — see Odds, and when they refresh
`/odds/comparison/`	Per-market × per-bookmaker grid — **requires Football Unlimited** (403 `bookmakers_not_entitled` without it; use `/api/v2/odds/` for the free consensus price)
`/polymarket/`	Prediction-market implied probabilities (0–1)
`/prediction/`	Model probabilities per market (same shape as `/api/v2/predictions/{id}/`)
`/metadata/`	Kit colours, fun facts, AI-generated match preview
`/availability/`	**Which of the above have data for this match**, in one call — see Ask before you fetch
`/broadcasts/`	TV broadcasters, filterable by `country_code`
`/social/`	Curated tweets/videos for the match (`type` filter)

```
curl -H "Authorization: Token YOUR_API_KEY" \
     "https://sports.bzzoiro.com/api/v2/events/223510/stats/"
```
copy

Head-to-head

```
GET /api/v2/events/{id}/h2h/
```
copy

Head-to-head is asked **of a match**, not of a pair of teams: the endpoint takes a fixture id and answers about the two clubs playing it. To get the history for two clubs with no fixture between them, find any past meeting first — `/api/v2/events/?team_id={a}` and pick one against club B — then ask this endpoint about it.

The aggregate is oriented to *that* fixture: `home_wins` means the club playing at home in the match you asked about.

```
{
  "total_matches": 10,
  "home_wins": 3, "draws": 5, "away_wins": 2,
  "home_goals": 11, "away_goals": 8,
  "avg_total_goals": 1.9,
  "home_win_rate": 0.3, "away_win_rate": 0.2,
  "recent_matches": [
    {
      "event_id": 213101,
      "date": "2026-04-04T16:30:00+00:00",
      "home": "Deportivo de La Coruña", "away": "Málaga CF",
```
copy

```
      "home_team_id": 1260, "away_team_id": 1259,
      "home_score": 1, "away_score": 1,
      "score": "1-1"
    }
  ]
}
```


Read the sides from the ids, not the names. `home` and `away` are the names as they were recorded on that match, and they get rewritten upstream — the same club shows up as "Deportivo de La Coruña", "Deportivo La Coruña" and "Deportivo de A Coruña" across three rows of one history. `home_team_id` and `away_team_id` are stable, and `event_id` links the row back to the full match.

`home_score` and `away_score` are integers, and null on a meeting with no result recorded. Parsing the `score` string instead turns that case into `0-0`, which reads as a real goalless draw.


```
{
  "event_id": 223510,
  "xg_estimated": false,
  "stats": {
    "home": {
      "ball_possession": 54, "total_shots": 13, "shots_on_target": 5,
      "shots_off_target": 4, "blocked_shots": 3, "hit_woodwork": 1,
      "xg": { "actual": 1.62, "estimated": false }, "...": "..."
    },
    "away": { "ball_possession": 46, "total_shots": 9, "shots_on_target": 3, "...": "
    "first_half": { "home": { "...": "..." }, "away": { "...": "..." } },
    "second_half": { "home": { "...": "..." }, "away": { "...": "..." } }
  },
  "shotmap": [
    {
      "player_id": 40112, "home": true, "min": 23, "type": "goal",
      "sit": "assisted", "body": "left-foot",
      "pos": { "x": 88.2, "y": 44.1, "z": 0 },
      "gm": { "x": 0, "y": 52.1, "z": 30.4 }, "gml": "low-centre",
      "xg": 0.34, "xgot": 0.61, "xg_estimated": false
    }
  ],
  "momentum": [ { "m": 1, "v": 12 }, { "m": 2, "v": -4 } ],
  "average_positions": {
    "home": [ { "player_id": 40112, "name": "B. Saka", "pos": "F", "x": 61.5, "y": 38
  },
  "xg_per_minute": [
    { "m": 23, "xg_home": 0.34, "xg_away": 0.0, "cum_home": 0.34, "cum_away": 0.12,
      "estimated": false }
  ]
}
```


Reading the pieces: `pos` is the shot's origin on a 0–100 × 0–100 pitch, `gm` where it crossed the goal line, `type` is one of `goal`, `save`, `miss`, `block`, `post`, and `xgot` is xG on target (`null` for off-target shots). In `momentum`, `m` is the minute and `v` the pressure value, positive for the home side. In `average_positions`, `n` is how many touches the average is built from. `xg_per_minute` carries both the per-minute (`xg_home` / `xg_away`) and running-total (`cum_home` / `cum_away`) series.

> **Note:** shot-level xG and average positions exist for covered competitions; for others the arrays are empty rather than missing — code defensively either way.

Measured xG and estimated xG

Not every xG in this response is a measured one. Where the shot feed ships no price, we estimate it from the shot's own geometry, and that estimate is served in the same `xg` field a measured value would be. It is our number, not the feed's, and it is calibrated differently — for anything that trains on or grades against xG the two are separate variables, not one.

It is not a rounding error at the edges. **Whole competitions are estimated end to end** — France Ligue 2, England League Two, Liga Portugal 2 and the NWSL have no measured shot xG at all, and just under half of all matches with a shotmap carry at least some xG of ours.

Four flags tell you which is which. They all answer the same question and take the same three values: `true` = ours, `false` = measured, `null` = there is no xG to qualify (a shoot-out attempt, or a shot with no usable geometry — those are served with `xg: null` too).

WHERE	FIELD	SCOPE
Response root	`xg_estimated`	`true` when *any* xG below is ours. The one field to read when deciding whether a match belongs in a training set
Each `shotmap` entry	`xg_estimated`	The per-shot truth; every other flag derives from it
Each `stats.*.xg` block	`estimated`	Per side, and per half in `first_half` / `second_half`
Each `xg_per_minute` bucket	`estimated`	`true` when any shot in that minute is ours

The team block and the shots can legitimately disagree. `stats.home.xg` comes from the team aggregate where there is one, and a few hundred matches carry a measured team total over shots we priced ourselves — so `estimated: false` on the team block next to `xg_estimated: true` on its shots is not a bug, it is the distinction a single per-match flag would lose. The root `xg_estimated` follows the shots.

Two matches with the same competition and date can differ: the flags are per shot and computed from what is actually stored, never inferred from the competition.

Shot counts: which number to trust

`stats.*.total_shots` and the `shotmap` array come from two different upstream feeds. We publish both as they arrive and deliberately do not reconcile them, so they can disagree on the same match. Rules for picking one:

- **The shotmap is the per-shot record — use it when you need a shot count.** The stats bag is a separate aggregate, useful for the ~40 other metrics in it that have no per-event equivalent.
- **Within the bag the identity is** `total_shots = shots_on_target + shots_off_target + blocked_shots + hit_woodwork`. `hit_woodwork` is its own bucket, not folded into the other three — that alone accounts for most apparent arithmetic failures.
- **The shotmap includes penalty-shootout attempts, the bag never does.** Those rows carry `sit: "shootout"` and minutes past 90 or 120. Filter them out before comparing the two, or a match decided on penalties will look like it has 10 extra shots. For the same reason **no xG we publish counts them**: `xg_per_minute`, the per-half `xg` blocks and the team totals (`stats.home.xg.actual`, `stats.away.xg.actual`, and `home_xg_live` / `away_xg_live` on the event) all skip shootout rows, and those rows are served with `xg: null`. So the per-shot values in `shotmap` sum to the team total in the same response. Where the upstream aggregate itself had counted the shootout, we serve the total rebuilt from the shotmap instead — on a match decided on penalties that is several xG lower than what this endpoint returned before 2026-08-16.
- **Shots on target** corresponds to shotmap rows with `type` in `goal` or `save`, again after dropping shootout rows.

Two caveats after all of that.

A stats bag on a just-finished match is the snapshot taken around the final whistle; upstream keeps correcting the feed for a few hours afterwards, and we re-read it once the match has settled. So a bag can lag its shotmap for a window after full time.

And a residual survives even then: measured over 800 team-sides, once both feeds have been re-read after the match settled, **11% still disagree on the shot count**, almost always by one, in either direction. That is upstream's own two feeds disagreeing with each other and no amount of re-reading fixes it. The internal arithmetic is far tighter — after the settle-window re-read, fewer than 1% of sides fail the `total_shots` identity above. If you need one number and cannot show two, take the shotmap.

Odds, and when they refresh

```
curl -H "Authorization: Token YOUR_API_KEY" \
     "https://sports.bzzoiro.com/api/v2/events/209535/odds/"
```


```
{
  "event_id": 209535,
  "odds": {
    "home_win": 1.17, "draw": 7.26, "away_win": 15.71,
    "over_15_goals": 1.17, "over_25_goals": 1.56, "over_35_goals": 2.42,
    "under_15_goals": 4.7, "under_25_goals": 2.35, "under_35_goals": 1.53,
    "btts_yes": 2.48, "btts_no": 1.49
  },
  "last_update_at": "2026-08-16T21:18:14Z",
  "next_update_at": "2026-08-17T01:18:14Z",
```

```
  "update_interval_seconds": 14400,
  "update_reason": "pre-match, more than a day out"
}
```

Decimal consensus prices across the bookmakers we cover — the eleven keys above are the whole set, and any of them is `null` when that market is not quoted for the match. For prices broken out per bookmaker rather than averaged, see the odds feed.

This shape is the full-time summary and does not carry everything we hold: total corners, clean sheet, win to nil and the half-time / second-half variants of 1X2, total goals and both-teams-to-score are all quoted, and reach you through `/odds/comparison/` or the odds feed rather than these eleven keys. Prices are **pre-match only** at the moment.

The four scheduling fields answer "when does this change?" without you having to poll and diff:

FIELD	MEANING
`last_update_at`	When we last **asked** about this match. Not when a price moved — see below
`next_update_at`	The **earliest** moment these odds may change
`update_interval_seconds`	How often this match is currently re-read
`update_reason`	Why it is on that interval, in words

Three things to know before you build on them.

`last_update_at` is an ask, not a change. It advances every time we re-read the match, whether or not any price moved, so it answers "have you looked recently" and not "has anything moved". Nothing in this payload answers the second question — for that you need the per-bookmaker rows on `/api/v2/odds/`, where `movement` and `previous_decimal_odds` carry the direction and the price it moved from.

`next_update_at` is a floor, not a promise. It is the earliest the odds may change; a re-read can run late, and on a busy evening it does. Treat a time that has passed as "due now", not as "we missed it". If it is already in the past because the match is overdue for a re-read, the field simply reports now.

The timestamps go `null`; `update_reason` never does. When nothing is scheduled — a match that finished more than a few hours ago, or one no bookmaker we read is quoting — `next_update_at` and `update_interval_seconds` are `null` and `update_reason` reads `"no further updates scheduled"`. `last_update_at` is additionally `null` when the match was never carried at all, and the `odds` block may still hold the last prices we saw. The `null` is deliberate: it says "we cannot tell you", which is honest, where a timestamp would be a time that never arrives. Branch on `update_reason`, which always has a value.

`update_interval_seconds` currently takes one of three values, and `update_reason` names each:

INTERVAL	`UPDATE_REASON`	WHEN
`900` (15 min)	`match is in play`	The match is live
`1800` (30 min)	`kick-off within 24 hours`	Kickoff is inside a day
`14400` (4 h)	`pre-match, more than a day out`	Everything further out

These are the intervals the schedule is built on; what we actually observe, including the tail, is in Refresh cadence.

Stats coverage varies

`/stats/` returns what the match actually reported, and that varies enormously: a fully covered league game carries around 44 statistics per side, a lower-tier friendly can carry four.

Eight keys are **always present** on each side, so you can read them without guarding:

`ball_possession`, `total_shots`, `shots_on_target`, `corner_kicks`, `fouls`, `yellow_cards`, `red_cards`, `offsides`

When a match did not report one of them the value is **`null`** — never a missing key, and never `0`. The distinction matters: `0` would assert that nothing happened, and on a thinly covered fixture that would be us inventing a fact. `null` says we do not know.

Everything beyond those eight (`expected_goals`, `big_chances`, `tackles`, `recoveries`, …) appears only when the match reported it, so read it defensively — check the key before you use it.

Measured on 2026-08-22 across 214 finished matches from the previous three days: 87% were missing at least one of the eight before this guarantee existed. `red_cards` was absent in 149 of them, `total_shots` in 51.

When a match is postponed

A postponed fixture keeps its original row and its original kick-off. We do not move it, because that is what upstream holds and because a date you already processed should not change under you.

What you want is the link to where it went:

```
{"id": 220285, "event_date": "2026-10-02T01:00:00+00:00",
 "status": "postponed", "replaced_by": 605456}
```

copy

`replaced_by` is the id of the rescheduled fixture. **It is filled as soon as the replacement is scheduled, not after it is played** — which is the point: if you only learn where a match moved once it has been played, you have already processed it on the wrong day.

The replacement is matched on the identity the source gives both rows, plus the same home/away sides and a six-month window. The reverse fixture shares that identity with the sides swapped, so it is never mistaken for a replay.

If you list a season with `?season_id=`, postponed fixtures that have a replacement are left out and the replacement is included instead — one row per fixture, on the date it will actually be played.

Which line is the main line

A match publishes eight or ten Asian handicaps and as many totals, and none of them used to be marked, so the response told you every line without telling you which one is *the* handicap for the match.

`/api/v2/events/{id}/odds/comparison/` now carries `main_lines`:

```
{"main_lines": {"asian_handicap": 0.0, "over_under_25": 2.5,
                "total_corners": 9.5}}
```

copy

The rule is the market's, not ours: the main line is the one where the two best prices sit closest together, because that is the handicap the market treats as a coin flip. On Netherlands v Germany that is level (2.06 / 1.89), not `-0.25` (2.40 / 1.68) and certainly not `-2.5` (9.60 / 1.07).

It is computed from the **best price on each side** rather than from any one bookmaker, so the answer does not change depending on who you ask — your book may not even quote the main line.

A market with no line is absent from the map, and so is a line nobody priced on both sides: half a market cannot be a coin flip.

Fixture change feed

`GET /api/v2/fixtures/changes/` returns only what moved about scheduled matches, so you can stop re-fetching fixture lists and diffing them yourself.

PARAM	TYPE	DESCRIPTION
`since`	datetime	ISO-8601 instant. Default 24 hours ago, capped at 7 days back.
`league_id`	int	Only changes to matches in this competition.
`change`	string	Comma-separated subset of `kickoff,status,venue,broadcast_added`.
`limit`	int	1–500, default 100.

```
{"since": "2026-09-25T09:00:00Z", "until": "2026-09-26T09:00:00Z",
 "count": 2, "truncated": false,
 "changes": [
   {"change": "kickoff", "changed_at": "2026-09-25T14:02:11Z", "event_id": 212581,
    "league_id": 64, "home_team": "Netherlands", "away_team": "Germany",
    "event_date": "2026-09-24T20:45:00+00:00",
    "old_value": "2026-09-24T18:45:00+00:00", "new_value": "2026-09-24T20:45:00+00:00
   {"change": "status", "changed_at": "2026-09-25T16:30:04Z", "event_id": 224136,
    "league_id": 17, "home_team": "Luton", "away_team": "Millwall",
    "event_date": "2026-09-27T14:00:00+00:00",
```

```
    "old_value": "notstarted", "new_value": "postponed"}
]}
```

Normal progress is not a change. A match going `notstarted` → `inprogress` → `finished` never appears. Only transitions in or out of a disrupted state do — postponed, cancelled, abandoned, suspended, awarded, walkover — because putting the routine ones in would bury the one you are polling for.

`broadcast_added` entries carry an empty `old_value`. A channel appearing is an addition, not a move, so there is no previous value and we do not invent one.

To poll without gaps: take `until` from the response and send it as the next `since`. When `truncated` is `true` the limit cut the window short — do *not* move `since` to `until`, re-ask from the last `changed_at` you received, or you will skip whatever was cut.

A fixture row only carries its current kickoff and status, so a move is only visible if it was written down as it happened. Recording started the day this endpoint shipped; nothing before that can be recovered.

Ask before you fetch

Most of the endpoints above return an empty body rather than an error when a match has nothing for them, which is the right behaviour but makes "does this match have corners?" cost a call to find out — and a call per sub-resource per match adds up fast.

`/api/v2/events/{id}/availability/` answers for all of them at once:

```
{"event_id": 212581, "status": "finished", "lineup_status": "confirmed",
 "available": {"stats": true, "lineups": true, "incidents": true,
               "shotmap": true, "xg": true, "momentum": true,
               "average_positions": true, "player_stats": true,
               "prediction": true, "broadcasts": true, "odds": true},
 "stat_keys": ["ball_possession", "corner_kicks", "yellow_cards", "..."],
 "odds_markets": ["1x2", "asian_handicap", "over_under_25", "total_corners"]}
```

`stat_keys` lists only the statistics this match actually carries — the eight always-present keys appear only when they are non-null — so it is the precise answer to whether corners or cards are there. `odds_markets` uses the same codes `/api/v2/odds/?market=` accepts.

Every value is computed from the rows it describes, at the moment you ask. There is deliberately no stored `has_corners` column: a stored flag drifts out of step with the data it describes, and a flag that says yes when the answer is no is worse than the empty call it saves, because you stop checking.

`false` means the call would come back empty, not that it would fail. The endpoints still answer 200.

When a lineup exists

`/lineups/` always answers. When there is nothing to serve it returns a **200 with `lineups: null`**, not a 404 — so an integration that reads "empty" as an error will call a perfectly normal fixture broken. The field to branch on is `lineup_status`:

LINEUP_STATUS	LINEUPS	BETA	WHAT IT IS
`confirmed`	XI + bench per side	`false`	The published teamsheet
`predicted`	XI + bench per side	`true`	An AI-predicted XI, with `confidence` per side
`unavailable`	`null`	`false`	Neither exists for this match right now

Which one you get depends mostly on how far the match is from kickoff:

WHEN	USUALLY
More than ~14 days out	`unavailable` — the predictor has not reached it yet
~14 days to kickoff	`predicted`, refreshed every few hours
The last hour before kickoff	flips to `confirmed` once the teamsheet is published — we start looking 75 minutes out, and official sheets typically land inside the final 45

WHEN	USUALLY
Live or finished	`confirmed` if we have it, otherwise `unavailable` — never `predicted`

Measured on 2026-08-15 across matches not yet started, a predicted XI exists for 100% of them within 3 days of kickoff, 99.8% within 7 days, 96% within 14 and 42% within 30. So a fixtures widget looking a month ahead sees `unavailable` on more than half of what it renders — that is the horizon, not a gap.

The empty shape, in full:

```
{
  "event_id": 222647,
  "lineup_status": "unavailable",
  "beta": false,
  "lineups": null,
  "unavailable_players": null,
  "updated_at": null
}
```

copy

Coverage on past matches

Confirmed teamsheets thin out going backwards. Of finished competitive matches — club friendlies excluded — measured on 2026-08-15: **98.6% of 2026 and 69% of 2025** carry one, with no month since August 2025 below 91%; but **8% of 2024, 3% of 2023 and effectively none before 2022**. Club friendlies are patchy in any year — 42% over the last 30 days.

Those older percentages are being filled and the numbers above will be out of date. The archive fill originally brought back results and in-match events without teamsheets; a pass now running is going back over those matches and attaching the sheet where one exists. It is a large backlog, so expect the figures to move rather than jump.

Where a match still has no teamsheet after that, it is because there is none to be had rather than because we stopped looking — and that is a per-competition answer, not a per-season one: two matches from the same year, one in a top division and one in a regional cup, routinely differ. We do not publish a year before which lineups are unavailable, because there isn't one.

`/football-coverage/` has the live per-competition picture, Lineups included — that is the number to trust over the ones on this page.

If squads look right but match lineups do not, that is the expected split rather than a bug: `/api/v2/teams/{id}/squad/` is the club's registered squad and is independent of any match, so it answers in full for an archive fixture that has no teamsheet at all.

Predicted benches

When `lineup_status` is `predicted`, `substitutes` is a **predicted bench**: the players the model actually rates as likely to be named, ranked by `ai_score` (highest first) and capped at 12, the widest bench any competition we cover puts on a teamsheet.

It used to be the whole registered squad minus the predicted XI — a median of 21 players and as many as 64 — which was not a bench anybody had predicted. Players the model scores at zero are now dropped rather than listed.

Two consequences worth coding for:

- **The bench can be short.** Where the model has little to go on it may name seven, or fewer. That is the prediction, not a truncation — read `confidence` on the side alongside it.
- **Confirmed lineups are never trimmed.** Once `lineup_status` is `confirmed`, the bench is the published teamsheet, served whole.

For the full registered squad, use `/api/v2/teams/{id}/squad/` instead — that is a squad list and is not affected by any of this.

Typical patterns

- **Match center page:** detail + `/stats/` + `/incidents/` + `/lineups/` on load; then poll `/api/v2/events/live/?league_id=…` or use the WebSocket for updates.
- **Fixtures widget:** list with `season_id` + `status=upcoming`, group by `stage` **and then** `round_number` — round numbers restart in every stage, so grouping by the number alone merges a league-phase matchday with a knockout tie. See Competition structure.
- **Result + odds recap:** list with `status=finished` + `/odds/` per match.

← PREVIOUS
Overview

NEXT →
Leagues, seasons & standings

View this page as Markdown · Found a mistake? Tell us

© 2026 Bzzoiro LLC — sports.bzzoiro.com